

SmartAttend — Final Project Analysis

Network-Based Attendance System · Complete technical analysis

A deep, internals-level walkthrough of the system. This document assumes no prior knowledge of the codebase. Read it top to bottom and you will understand what every service does, how a single Wi-Fi association turns into a database row, why each design decision was made, and where the system's real limits are.

Companion documents: `README.md` for a quick overview and setup, `docs/api-contract.md` for request/response payload examples.

Table of contents

- 1. The problem and the core idea
- 2. System architecture
- 3. The presence detection pipeline
- 4. Layer 2 networking: what ARP actually gives us
- 5. Proximity proof: BSSID and signal strength
- 6. The confidence scoring algorithm
- 7. The confidence gate: refusing to mark
- 8. Database schema
- 9. Backend structure, class by class
- 10. The two WebSocket channels
- 11. Complete API reference
- 12. Frontend architecture
- 13. Security model
- 14. Configuration, profiles and environments
- 15. Running the system
- 16. Testing
- 17. The demo script
- 18. Design decisions and tradeoffs
- 19. Known limitations and threat model
- 20. File-by-file map
- 21. Networking glossary

1. The problem and the core idea

Roll-call attendance wastes lecture time and is trivially defeated by proxy answers. Biometric and RFID systems solve that but need dedicated hardware in every room.

SmartAttend's premise: **the students are already carrying a uniquely identifiable network device, and the campus network already knows it is there.** When a phone associates with the classroom Wi-Fi, its network interface card announces a globally unique MAC address on the local network segment. If we can map that MAC to a student and prove the device is physically in the room, we have attendance with zero extra hardware.

The project therefore has two halves, and it is important to understand that the second half is what makes it a *networking* project rather than a CRUD app:

Half	Question it answers	Mechanism
Identity	<i>Whose device is this?</i>	MAC address to student mapping, admin-approved
Proximity	<i>Is it actually in this room?</i>	Access point BSSID match plus signal strength

Identity alone is not attendance. A student sitting in the canteen on campus Wi-Fi is identified perfectly and is not in the lecture. Solving only the first half produces a system that marks the whole campus present, which is why the proximity half exists and why the backend is willing to refuse a mark.

2. System architecture

Three independent processes, each with a single clear responsibility:



Why three processes instead of one

The ARP table is an operating-system resource. Reading it requires shelling out to platform-specific commands (`arp -a` , `iw` , `wdutil` , `netsh`) and parsing text output that differs per OS. Isolating that in a small Node service means:

- The scanner can run on a different machine from the backend — specifically, on a host inside the classroom or on the access point itself, which is exactly where you need to be to observe per-client signal strength.
- The Spring backend stays portable and testable, with no shell dependencies.
- The scanner can be replaced (by an SNMP poller against a real controller, or a RADIUS accounting listener) without touching business logic.

Ports at a glance

Port	Service	Protocol	Purpose
3001	presence-service	HTTP	Status and simulator control
3002	presence-service	WebSocket	Event stream to the backend
8080	backend	HTTP + WebSocket	REST API, STOMP endpoint, Swagger UI
3306	MySQL	TCP	Persistence
5173	frontend	HTTP	Vite dev server (port 80 inside Docker)

3. The presence detection pipeline

This is the heart of the system. Follow one device from association to database row.

Step 1 — Scan cycle (`presence-service/src/index.js` , `runScanCycle`)

Every `SCAN_INTERVAL_MS` (default 10 000 ms) three things happen in parallel:

```
const [devices, localWireless, stationSignals] = await Promise.all([
  scanNetwork(),          // parse `arp -a`
  getLocalWireless(),     // which AP am I on?
  getStationSignals(),    // per-client RSSI, if I am the AP
]);
```

Each ARP entry is then enriched with wireless context:

```
const enriched = devices.map((d) => ({
  ...d,
  bssid: localWireless.bssid || null,
  rssi: stationSignals.has(d.mac) ? stationSignals.get(d.mac) : null,
}));
```

Note the honesty in that second line: `rssi` is only attached when genuine per-station data exists. The scanner never substitutes its own signal strength for a student's, because that would be a fabricated measurement.

Step 2 — Diffing

The service keeps `previousDevices` and `currentDevices`. New MACs produce `presence_event` messages with `status: "connected"`. MACs that vanished produce `status: "disconnected"`. Every sixth cycle (so roughly once a minute) a full `snapshot` is broadcast as a safety net against missed events.

Disconnect events are deliberately **not** used to un-mark attendance. A phone that sleeps its Wi-Fi radio would otherwise erase a legitimate mark.

Step 3 — Transport

Events go out as JSON over the raw WebSocket on 3002:

```
{
  "type": "presence_event",
  "mac": "AA:BB:CC:11:22:01",
  "ip": "192.168.1.101",
  "bssid": "A4:83:E7:C0:FF:EE",
  "rssi": -47,
  "status": "connected",
  "timestamp": "2026-09-17T18:03:12.105Z"
}
```

Step 4 — Ingestion (`PresenceIngestionService`)

A `Java-WebSocket` client connects on startup (`@PostConstruct`) and reconnects every 5 seconds on failure, so service start order does not matter. It parses the frame and calls the attendance service. `textOrNull` and `intOrNull` helpers convert missing or blank JSON fields to Java `null` rather than the string `"null"` or `0`, which matters because a missing RSSI must be distinguishable from a reading of zero.

Step 5 — Attendance decision (`AttendanceService.processPresenceEvent`)

The full decision sequence:

1. **Record the sighting.** `PresenceSightingStore` keeps the last `(mac, ip, seenAt)` in a `ConcurrentHashMap`. Comparing the previous sighting to the current one yields `ipChangedRecently`: the same MAC on a different IP within 120 seconds, which hints at address cloning.
2. **Resolve identity.** `deviceService.findStudentByApprovedMac(mac)` looks for a device row that is both registered and `approved = true`. Unknown or unapproved MACs are dropped silently — a visitor's laptop must never create attendance.
3. **Pick candidate sessions.** All active sessions, sorted by `startTime` descending, so the most recently started lecture wins.
4. **Skip duplicates.** `existsBySessionAndStudent` prevents a second row for the same student in the same session.
5. **Cross-session guard.** If the student was marked in a *different* session within the last 10 minutes, skip. This stops a single device from being counted in two simultaneous lectures.
6. **Score confidence** against the session's classroom (section 6).
7. **Gate.** Below `app.presence.min-confidence`, refuse and broadcast why. Otherwise persist an `AUTO` record and broadcast the mark (section 7).
8. **Break.** Either outcome stops the loop, so one sighting affects at most one session.

Step 6 — Live push to the browser

The backend publishes to `/topic/session/{sessionId}` over STOMP. The faculty dashboard is subscribed and updates the radar and tables with no polling.

4. Layer 2 networking: what ARP actually gives us

Understanding this section is essential to discussing the project honestly.

ARP (Address Resolution Protocol, RFC 826) resolves a Layer 3 IPv4 address to a Layer 2 MAC address inside a single broadcast domain. When host A wants to send to `192.168.1.101` on its own subnet, it broadcasts an ARP request ("who has 192.168.1.101?"); the owner replies with a unicast ARP reply carrying its MAC. The result is cached in the ARP table.

`arp -a` prints that **cache**, which produces three consequences the code lives with:

1. **It is passive and incomplete.** The cache only holds entries for hosts this machine has recently exchanged frames with. A classroom of 60 phones that never talk to the scanner host will largely not appear. Populating the table properly needs active discovery — an ARP sweep or ping sweep across the subnet — which this build does not yet do.
2. **Entries age out.** Timeouts vary by OS, from roughly 60 seconds to 20 minutes. "Currently present" is therefore a fuzzy notion, which is precisely why `PresenceConfidence` scores recency (`SCAN_WINDOW_SECONDS = 30`) instead of trusting the table blindly.
3. **It is limited to one broadcast domain.** The scanner only sees devices on its own subnet. Devices behind a router are invisible. For a campus this means one scanner per VLAN, which is an argument for the scanner living on the access point.

Why MAC and not IP

IP addresses are leased by DHCP and rotate. A MAC address is burned into the NIC and is stable for the device, which makes it the correct identity key. IP is still recorded because it is useful evidence: the campus-subnet check and the clone-detection heuristic both rely on it.

Parsing

`scanner.js` handles two output formats. On Linux and macOS the line looks like `? (192.168.1.1) at aa:bb:cc:dd:ee:ff [ether] on en0` ; on Windows it is a three-column table with hyphen-separated MACs. Both parsers normalise to uppercase colon form and discard the broadcast MAC `FF:FF:FF:FF:FF:FF` and `.255` broadcast addresses.

5. Proximity proof: BSSID and signal strength

Two new pieces of evidence separate "on the campus LAN" from "in this room".

BSSID is the MAC address of the specific access point radio a client is associated with. Because APs are physically installed in rooms, the BSSID is effectively a room identifier. Registering a classroom's BSSID turns it into an attendance geofence.

RSSI (Received Signal Strength Indicator) is measured in dBm and is always negative. Roughly: -30 dBm is touching the AP, -50 is the same room, -70 is one wall away, -85 is barely usable. A configurable per-room cutoff (default -70 dBm) distinguishes people in the room from people in the corridor on the same AP.

Three detection modes, in descending fidelity

`presence-service/src/wireless.js` tries each and degrades gracefully:

Mode	Command	What you get
AP mode	<code>iw dev <iface> station dump</code>	True per-client MAC and RSSI. The real thing.
Observer mode	<code>iw dev <iface> link</code> (Linux), <code>wdutil info</code> or <code>airport -I</code> (macOS), <code>netsh wlan show interfaces</code> (Windows)	The AP the scanner itself is on, used as the room's AP. No per-client RSSI.
Declared mode	<code>CLASSROOM_BSSID</code> environment variable	Operator states the room's AP manually.

If all three fail, `getLocalWireless()` returns `{ bssid: null, source: 'unavailable' }` and scoring falls back to network-only evidence with an explicit "room presence unverified" reason. Nothing crashes and nothing is fabricated.

Two platform notes worth knowing: modern macOS redacts the BSSID from `wdutil info` unless it runs with `sudo`, and `airport` was removed in macOS 14.4, so a Mac scanner will usually report `unavailable` . Windows reports signal as a quality percentage, which the code maps to approximate dBm with `percent / 2 - 100` .

6. The confidence scoring algorithm

`PresenceConfidence` (in `backend/.../presence/`) is a deliberately **pure function** — no Spring annotations, no database, no clock of its own. Everything it needs arrives in a `Signals` record and the result is a `Result(score, level, reasons)` . That design is why it can be unit-tested exhaustively without booting an application context.

Inputs

```
public record Signals(  
    boolean macApproved,           // device is registered and approved  
    String ip,                     // IPv4 from the ARP table  
    List<String> campusCidrs,      // configured campus subnets  
    Instant lastSeen,              // when this MAC was last observed  
    Instant now,                   // evaluation time  
    boolean ipChangedRecently,    // same MAC, different IP, within 120 s  
    boolean duplicateMacObserved,  
    String observedBssid,          // AP the device is on  
    String expectedBssid,          // AP registered for this classroom  
    Integer rssiDbm,               // signal strength, may be null  
    int minRssiDbm)                // this room's cutoff
```

Two modes

The algorithm branches on whether the session is bound to a classroom (`expectedBssid` present). This is the single most important design point in the scoring: **when the room is known, network evidence alone is mathematically incapable of reaching HIGH confidence.**

Signal	Room-bound weight	Unbound weight
MAC registered and approved	+30	+40
Seen within the 30 s scan window	+15	+20
IP inside a campus subnet	+15	+20
Associated with the classroom AP	+30	not evaluated
Signal at or above the room cutoff	+10	not evaluated
Associated with a <i>different</i> AP	-35	not evaluated
Signal below the room cutoff	-15	not evaluated
Source IP changed recently	-20	-20
Duplicate MAC observed	-25	-25

The score is clamped to 0–100 and converted to a level:

Score	Level
75–100	HIGH
50–74	MEDIUM
0–49	LOW

Worked examples

Scenario	Arithmetic	Result
In the room, strong signal	30 + 15 + 15 + 30 + 10	100, HIGH — marked
Room-bound, no AP data available	30 + 15 + 15	60, MEDIUM — marked, but not claimed as room-verified
On campus, wrong AP (the corridor case)	30 + 15 + 15 - 35	25, LOW — refused
Correct AP but very weak signal	30 + 15 + 15 + 30 - 15	75, HIGH — marked, near the boundary
Unbound session, everything good	40 + 20 + 20	80, HIGH — legacy behaviour preserved

The unbound row matters for backward compatibility: sessions created without a room behave exactly as they did before proximity existed, so nothing regresses.

Reasons

Every branch appends a human-readable string. They are joined with `;` into `attendance_records.confidence_reasons` and sent to the UI as an array. This is what lets the faculty screen answer "why was this student not marked?" without anyone reading logs:

```
MAC is registered and approved; Seen in current ARP scan window; IP is inside campus subnet; Associated with a different access point (A4:83
```

Supporting helpers

- `normalizeBssid` uppercases and converts hyphens to colons, so `a4-83-e7-c0-ff-ee` and `A4:83:E7:C0:FF:EE` compare equal.
 - `ipInCidr` / `ipv4ToLong` implement subnet matching by converting the address to a 32-bit integer and comparing under a prefix mask. Malformed
- input returns `false` rather than throwing, so a garbled ARP line cannot break the pipeline.
 - `manualMark()` returns a fixed `55, MEDIUM, "Marked by faculty (no live ARP proof)"`. Faculty overrides are trusted but explicitly flagged as lacking network evidence.

7. The confidence gate: refusing to mark

When the score falls below `app.presence.min-confidence` (default 50), the backend creates **no database row at all**. Instead:

```
if (confidence.score() < minConfidence) {
    broadcastRejection(session.getId(), student, mac, ip, bssid, rssi, confidence);
    break;
}
```

A `presence_rejected` message goes to the session topic carrying the student, MAC, AP, signal and the full reason list. The faculty UI renders these in a red "Seen on the network, not marked" table and as hollow red rings on the radar.

This is an intentional stance rather than a convenience. Recording a low-confidence row would leave a defensible-looking attendance entry backed by weak evidence, and someone downstream would inevitably treat it as fact. Refusing to record, while telling the operator exactly what was seen and why it was rejected, keeps the attendance table trustworthy and puts the human in the loop for edge cases. The faculty member can still mark the student manually, and that record is honestly labelled `MANUAL` at MEDIUM confidence.

8. Database schema

Five tables. Hibernate generates them from the entities with `ddl-auto=update` on MySQL, so adding a column never drops data.

users

Column	Type	Notes
<code>id</code>	BIGINT PK	auto-increment
<code>name</code>	VARCHAR	not null
<code>email</code>	VARCHAR	not null, unique — the login identity and JWT subject
<code>password</code>	VARCHAR	BCrypt hash, never plaintext
<code>role</code>	VARCHAR	enum as string: <code>STUDENT</code> , <code>FACULTY</code> , <code>ADMIN</code>
<code>student_id</code>	VARCHAR	unique, nullable — the PRN; null for staff
<code>created_at</code>	DATETIME	set in <code>@PrePersist</code>

devices

Column	Type	Notes
<code>id</code>	BIGINT PK	
<code>mac_address</code>	VARCHAR	not null, unique globally — one MAC cannot serve two students
<code>device_name</code>	VARCHAR	free text, e.g. "Demo iPhone"
<code>student_id</code>	BIGINT FK → <code>users.id</code>	not null, lazy
<code>approved</code>	BOOLEAN	not null — only <code>true</code> participates in detection
<code>created_at</code> , <code>approved_at</code>	DATETIME	audit trail

Business rule enforced in `DeviceService.registerDevice`: a student may hold only **one approved device at a time**. Registering a second is rejected until the first is removed, which prevents one student blanketing the room with devices.

classrooms

Column	Type	Notes
<code>id</code>	BIGINT PK	
<code>name</code>	VARCHAR	not null, e.g. "CNT Lab 401"
<code>bssid</code>	VARCHAR	not null, unique — the AP MAC, normalised uppercase
<code>ssid</code>	VARCHAR	nullable, informational
<code>min_rssi_dbm</code>	INT	defaults to -70 via <code>DEFAULT_MIN_RSSI_DBM</code>
<code>created_at</code>	DATETIME	

sessions

Column	Type	Notes
<code>id</code>	BIGINT PK	
<code>course_name</code>	VARCHAR	not null
<code>course_code</code>	VARCHAR	nullable
<code>faculty_id</code>	BIGINT FK → <code>users.id</code>	not null
<code>classroom_id</code>	BIGINT FK → <code>classrooms.id</code>	nullable — null means unbound, network-only scoring
<code>start_time</code>	DATETIME	not null; <code>@PrePersist</code> only fills it when null, so seeders can backdate
<code>end_time</code>	DATETIME	nullable until ended
<code>active</code>	BOOLEAN	not null; one active session per faculty is enforced in the service

attendance_records		
Column	Type	Notes
id	BIGINT PK	
session_id	BIGINT FK	not null
student_id	BIGINT FK	not null
marked_at	DATETIME	not null
status	VARCHAR	AUTO, MANUAL, or OVERRIDE
device_mac, device_ip	VARCHAR	evidence captured at mark time
observed_bssid	VARCHAR	which AP saw the device
signal_dbm	INT	nullable — null means no per-station reading
confidence_score	INT	0–100
confidence_level	VARCHAR	HIGH / MEDIUM / LOW
confidence_reasons	VARCHAR(1000)	; -joined audit trail

Unique constraint on (session_id, student_id) — the database itself guarantees one record per student per session, so even a race in the ingestion path cannot double-mark.

Relationships

```

users 1 — n devices
users 1 — n sessions      (as faculty)
users 1 — n attendance_records (as student)
classrooms 1 — n sessions  (optional)
sessions 1 — n attendance_records

```

All @ManyToOne associations are FetchType.LAZY to avoid loading the whole graph on every query.

9. Backend structure, class by class

Package root: com.smartattendance.

model/ — JPA entities

User, Device, Classroom, Session, AttendanceRecord, and the Role enum. Lombok @Getter @Setter @Builder removes boilerplate; @PrePersist hooks set timestamps.

repository/ — Spring Data JPA

Interfaces only; Spring generates implementations. Beyond derived queries like findByMacAddressAndApproved, AttendanceRecordRepository carries three explicit JPQL queries for date-range and course filtering.

presence/ — the networking brain

Class	Responsibility
PresenceConfidence	Pure scoring function plus CIDR and BSSID helpers
PresenceSightingStore	In-memory last-seen map for recency and IP-clone detection
MacVendor	OUI prefix lookup returning a vendor hint such as Apple or Samsung

MacVendor deserves a note: the first three octets of a MAC are the Organizationally Unique Identifier assigned to the manufacturer. A static map of roughly two dozen prefixes covers Apple, Samsung, Google, Huawei, Realtek, Raspberry Pi, VMware and Parallels, turning a raw address into a readable vendor. This makes the approval queue far easier to audit and is a genuine Layer 2 detail rather than decoration. The demo prefix AA:BB:CC is mapped to "Demo Classroom Device" so simulated devices are obvious, and the demo BSSID A4:83:E7 is a real Apple OUI.

service/ — business logic

Class	Responsibility
AuthService	Register and login, BCrypt hashing, JWT issuance
DeviceService	Registration, approval, bulk approval, MAC-to-student resolution
ClassroomService	Room CRUD with BSSID normalisation and duplicate rejection
SessionService	Start/end sessions, one-active-per-faculty rule, room binding
AttendanceService	The decision engine: scoring, gating, persistence, broadcasts, analytics
PresenceIngestionService	WebSocket client consuming the Node event stream
PresenceGateway	HTTP client calling the Node REST API (/presence/current , /presence/simulate)

`AttendanceService` is the largest class and also computes the read models: `getStudentDashboard` (84-day heatmap, attendance percentage, live session banner) and `getAnalytics` (daily series, per-course totals, method split, average confidence, and students below 75 % flagged as defaulters).

controller/ — REST layer

`AuthController`, `StudentController`, `FacultyController`, `AdminController`, `ReportController`. All return `ApiResponse{success, message, data}` except the two auth endpoints, which return `AuthResponse` unwrapped — a small inconsistency worth knowing when writing a client.

config/

- `SecurityConfig` — filter chain, role rules, BCrypt bean, stateless sessions
- `WebSocketConfig` — STOMP broker at `/ws`, topic prefix `/topic`, app prefix `/app`
- `CorsConfig` — origins from `app.cors.allowed-origins`, credentials enabled
- `DemoDataSeeder` — idempotent demo data (see below)

security/

- `JwtTokenProvider` — HMAC-SHA signing, 24-hour expiry, email as subject
- `JwtAuthenticationFilter` — `OncePerRequestFilter` reading the `Bearer` header and populating the `SecurityContext`
- `CustomUserDetailsService` — loads a user by email for Spring Security

The seeder

`DemoDataSeeder` is a `CommandLineRunner` that is **idempotent and non-destructive**. It checks existence before every insert (`findByEmail`, `existsByBssid`, `findByMacAddress`), so it can run against a populated production database without touching a single existing row. It creates:

- One admin, one faculty, and ten students (`student1..10@smartattend.edu`), all with password `Demo@123` and PRNs `CS2024D01` — `CS2024D10`
- One approved device per student, `AA:BB:CC:11:22:01` through `...:0A`
- Two classrooms: **CNT Lab 401** on `A4:83:E7:C0:FF:EE` and **Lecture Hall 2 (adjacent)** on `A4:83:E7:11:22:33`

The MAC range and the two BSSIDs are shared with the simulator's defaults in `presence-service/src/config.js`. That alignment is what makes the demo work end to end; change one side and you must change the other.

10. The two WebSocket channels

A common source of confusion: there are **two different WebSocket systems** in this project and they are not the same technology.

	presence-service → backend	backend → browser
Endpoint	<code>ws://localhost:3002</code>	<code>http://localhost:8080/ws</code>
Protocol	Raw WebSocket, bare JSON frames	STOMP over SockJS
Server	<code>ws</code> npm package	Spring <code>@EnableWebSocketMessageBroker</code>
Client	<code>Java-WebSocket</code> 1.5.7	<code>@stomp/stompjs</code> + <code>sockjs-client</code>
Addressing	Single broadcast to all clients	Subscription to <code>/topic/session/{id}</code>
Message types	<code>presence_event</code> , <code>snapshot</code>	<code>attendance_marked</code> , <code>presence_rejected</code>
Authentication	None	None on the endpoint (see section 13)

STOMP is used on the browser side because it provides topic-based subscription, so the backend can address updates to one session rather than broadcasting everything to every connected dashboard. SockJS wraps it with fallbacks for environments that block raw WebSockets.

Message shapes

`attendance_marked` :

```
{
  "type": "attendance_marked",
  "sessionId": 1, "studentId": 4,
  "studentName": "Ananya Iyer", "studentRollNo": "CS2024D04",
  "mac": "AA:BB:CC:11:22:04", "ip": "192.168.1.104",
  "bssid": "A4:83:E7:C0:FF:EE", "rssi": -42,
  "status": "AUTO", "markedAt": "2026-09-17T23:35:02",
  "confidenceScore": 100, "confidenceLevel": "HIGH",
  "confidenceReasons": ["MAC is registered and approved", "..."],
  "vendor": "Demo Classroom Device"
}
```

`presence_rejected` carries the same identity and evidence fields plus `seenAt` , and notably **no** `markedAt` or record id, because nothing was stored.

11. Complete API reference

Base URL `http://localhost:8080/api` . Interactive docs at `http://localhost:8080/swagger-ui.html` (SpringDoc OpenAPI).

Public

Method	Path	Purpose
POST	<code>/auth/register</code>	Create account; returns JWT. Only STUDENT or FACULTY accepted
POST	<code>/auth/login</code>	Exchange credentials for a JWT

Student — requires role `STUDENT`

Method	Path	Purpose
GET	<code>/student/dashboard</code>	Devices, history, percentage, 84-day heatmap, live session
GET	<code>/student/attendance-history</code>	Flat attendance list with course details
POST	<code>/student/device/register</code>	Submit a MAC for approval
GET	<code>/student/devices</code>	Own devices with vendor hints
DELETE	<code>/student/devices/{deviceId}</code>	Remove own device (ownership checked)

Faculty — requires role `FACULTY`

Method	Path	Purpose
GET	<code>/faculty/classrooms</code>	Rooms available for binding
POST	<code>/faculty/session/start</code>	Start a session; body may include <code>classroomId</code>
POST	<code>/faculty/session/{id}/end</code>	End a session (ownership checked)
GET	<code>/faculty/session/active</code>	Active session plus current attendees
GET	<code>/faculty/session/{id}/live</code>	Attendance rows for a session
POST	<code>/faculty/session/{sessionId}/mark/{studentId}</code>	Manual mark by internal id
POST	<code>/faculty/session/{sessionId}/mark-by-prn</code>	Manual mark by PRN
GET	<code>/faculty/sessions</code>	Session history with room info
GET	<code>/faculty/presence/live</code>	Proxied live ARP view from the Node service
POST	<code>/faculty/presence/simulate</code>	Start the demo classroom

Admin — requires role ADMIN

Method	Path	Purpose
GET	/admin/devices/pending	Approval queue with student details
POST	/admin/devices/{id}/approve	Approve one device
POST	/admin/devices/{id}/reject	Delete a device
POST	/admin/devices/bulk-approve	Approve many: {"ids": [1,2,3]}
GET	/admin/users	All users
GET	/admin/classrooms	Registered rooms
POST	/admin/classrooms	Register a room; BSSID validated by regex
DELETE	/admin/classrooms/{id}	Remove a room

Reports — FACULTY or ADMIN

Method	Path	Purpose
GET	/reports/attendance?course=&date=	Filtered attendance report
GET	/reports/analytics	Aggregates for the charts and defaulter list

presence-service REST (port 3001, unauthenticated)

Method	Path	Purpose
GET	/presence/current	Devices, wireless context, scan count
GET	/presence/health	Uptime, client count, BSSID, detection source
GET	/presence/wireless	Wireless context and which method found it
POST	/presence/simulate	{count, intervalMs, outsideCount}
POST	/presence/simulate/clear	Drop simulated devices

12. Frontend architecture

React 19 with Vite 8, plain CSS (no framework), oxlint for linting.

Routing and guards

App.jsx composes AuthProvider → ToastProvider → Navbar + CommandPalette + routes. ProtectedRoute takes a roles array and redirects unauthorised users to their own home route.

Route	Role	Page
/	public	Landing (redirects logged-in users to their dashboard)
/login, /register	public	Login, Register
/student	STUDENT	StudentDashboard — heatmap, percent ring, live banner
/student/devices	STUDENT	StudentDevicesPage
/faculty	FACULTY	FacultyDashboard — the presence command center
/faculty/sessions	FACULTY	FacultySessionsPage — start session, pick room
/admin	ADMIN	AdminDashboard — device approval with bulk actions
/admin/classrooms	ADMIN	AdminClassroomsPage — the AP registry
/admin/users	ADMIN	AdminUsersPage
/reports	FACULTY, ADMIN	Reports — charts, CSV, PDF

State and transport

- AuthContext holds the user and mirrors token and user into localStorage so a refresh keeps the session.
 - api.js centralises axios. A request interceptor attaches Authorization: Bearer <token> ; a response interceptor catches HTTP
- 401, clears storage, and redirects to /login . API_BASE and WS_URL come from VITE_API_URL / VITE_WS_URL so Docker builds can point elsewhere.

 - ToastContext provides toast.success / toast.error for transient messages.

PresenceRadar — the signature component

A canvas animation driven by `requestAnimationFrame`. The important logic is that geometry encodes real measurements:

- **Radius from RSSI.** `radiusFromRssi` clamps signal to $-90 \dots -40$ dBm and maps it to $0.12 \dots 0.92$ of the radar radius, so stronger signal genuinely means closer to the AP at the centre. Range rings are labelled in dBm. Devices with no RSSI reading are parked on an outer ring derived from a hash, and that fallback is visually distinguishable rather than pretending to be a measurement.
- **Angle from a hash** of the student id, which is stable across renders but carries no meaning — bearing is not measurable from a single AP.
- **Colour from state.** Green for HIGH, amber for MEDIUM, grey for unregistered devices, and red hollow rings for refused devices.

Other components

`ConfidenceBadge` (HIGH/MEDIUM/LOW pill with score), `AttendanceHeatmap` (12x7 grid coloured by count), `PercentRing` (SVG progress ring), `CommandPalette` (%K navigation), `Navbar` (role-aware links with a mobile hamburger).

`utils/attendanceMath.js` holds `heatmapMax` and `attendancePercent` as pure functions so they can be unit-tested independently of React.

Reports

`recharts` renders the daily bar chart, per-course chart, and method pie chart. `jspdf` with `jspdf-autotable` produces a PDF; CSV export is built by hand from the same rows.

13. Security model

What is implemented

- **Password storage.** BCrypt via `BCryptPasswordEncoder` — salted and adaptive.
- **Authentication.** Stateless JWT, HMAC-SHA signed, 24-hour expiry, email as subject. `SessionCreationPolicy.STATELESS` means no server-side sessions.
- **Authorisation.** Path-prefix role rules in `SecurityConfig`: `/api/admin/**` requires ADMIN, `/api/faculty/**` requires FACULTY, `/api/student/**` requires STUDENT, `/api/reports/**` allows either FACULTY or ADMIN.
- **Privilege-escalation guard.** `AuthService.register` accepts only STUDENT or FACULTY. A self-registered ADMIN is impossible; the role silently falls back to STUDENT.
- **Ownership checks.** Ending a session, removing a device, and manual marking all verify the caller owns the resource rather than trusting the id in the URL.
- **CORS.** Explicit origin allowlist from configuration, not a wildcard.
- **Database integrity.** Unique constraints on email, MAC, BSSID, and `(session_id, student_id)` enforce invariants below the application layer.

Known weaknesses, stated plainly

These are real and belong in any honest write-up:

1. **The presence feed is an unauthenticated trust boundary.** The Node WebSocket server on 3002 accepts any client, and the backend marks attendance based on whatever arrives. Anyone who can reach that port can forge attendance for any approved MAC. The fix is an HMAC signature per event with a timestamp and nonce for replay protection, or mTLS.
2. **The STOMP endpoint is `permitAll`.** `/ws/**` is public and `setAllowedOriginPatterns("*")`, so anyone can subscribe to `/topic/session/{id}` and read student names, PRNs, and MAC addresses. This is a privacy leak as much as an authorisation gap.
3. **BSSID is self-reported by the scanner.** Proximity proof is only as trustworthy as the host reporting it. A compromised scanner can claim any BSSID. This is the same trust-boundary problem as point 1.
4. **MAC addresses are forgeable.** `sudo ifconfig en0 ether <target>` impersonates another student's device. `duplicateMacObserved` exists in the scoring inputs but is currently always passed `false`, so duplicate detection never fires. Credible defences would be DHCP fingerprinting (option 55), OUI-versus-fingerprint consistency checks, and a nonce challenge the real device must answer over HTTP.
5. **JWT in `localStorage`** is readable by any injected script, so an XSS flaw becomes full account takeover. `HttpOnly` cookies would be stronger.
6. **No rate limiting on `/api/auth/login`**, so password brute-forcing is unthrottled.
7. **H2 console is `permitAll` and `frameOptions` is disabled.** Acceptable for local development, unacceptable if the H2 profile ever ran somewhere reachable.
8. **The JWT secret has a hardcoded default** in `application.properties`, and `JwtTokenProvider.getSigningKey()` Base64-encodes the raw secret bytes only to immediately Base64-decode them — a no-op round trip that ends up using the raw bytes. Harmless but confusing, and it means secret strength depends entirely on the configured string.

14. Configuration, profiles and environments

Backend properties

Key	Default	Meaning
<code>server.port</code>	8080	HTTP port
<code>spring.datasource.url</code>	<code>\${DB_URL:...}</code> MySQL localhost	JDBC URL
<code>spring.jpa.hibernate.ddl-auto</code>	<code>update</code>	Additive schema migration; never drops
<code>app.jwt.secret</code>	<code>\${JWT_SECRET:...}</code>	HMAC signing key
<code>app.jwt.expiration-ms</code>	86400000	24 hours
<code>app.presence.ws-url</code>	<code>\${PRESENCE_WS_URL:ws://localhost:3002}</code>	Event stream source
<code>app.presence.rest-url</code>	<code>\${PRESENCE_REST_URL:http://localhost:3001}</code>	Simulator and live view
<code>app.presence.campus-subnets</code>	<code>10.0.0.0/8,172.16.0.0/12,192.168.0.0/16</code>	RFC 1918 ranges treated as campus
<code>app.presence.min-confidence</code>	50	The refusal threshold
<code>app.cors.allowed-origins</code>	localhost 5173 and 3000	CORS allowlist
<code>springdoc.swagger-ui.path</code>	<code>/swagger-ui.html</code>	API docs

Every environment-sensitive value uses the `${VAR:default}` form, so the application starts with no environment file at all — useful for graders who just want it to run.

Profiles

- default** — MySQL, `ddl-auto=update`, data persists across restarts. `/h2-console`. **Everything is wiped on every restart**, which makes it perfect for clean demos and wrong for real data.
- h2** (`--spring.profiles.active=h2`) — in-memory H2 in MySQL compatibility mode with `ddl-auto=create-drop` and the console at

presence-service environment

Variable	Default	Meaning
<code>SCAN_INTERVAL_MS</code>	10000	Poll period
<code>REST_PORT</code> / <code>WS_PORT</code>	3001 / 3002	Listen ports
<code>CLASSROOM_BSSID</code>	unset	Declare the room's AP when auto-detection cannot
<code>DEMO_ROOM_BSSID</code>	<code>A4:83:E7:C0:FF:EE</code>	Simulator in-room AP; must match the seeded classroom
<code>DEMO_CORRIDOR_BSSID</code>	<code>A4:83:E7:11:22:33</code>	Simulator out-of-room AP

Frontend environment

`VITE_API_URL` and `VITE_WS_URL`, baked in at build time (see `frontend/.env.example`).

Docker

`docker-compose.yml` defines four services with correct ordering: MySQL exposes a `mysqladmin ping` healthcheck, and the backend waits on `condition: service_healthy` before starting, so it never races the database. Service names double as DNS hostnames, which is why the backend's URLs become `ws://presence:3002` and `jdbc:mysql://mysql:3306/...`. A named volume `smartattend-mysql` persists data. The frontend is built with the API URLs as build arguments and served on container port 80, published as 5173.

15. Running the system

Prerequisites

JDK 17, Node 18+, MySQL 8 (or use the H2 profile), Maven (the `mvnw` wrapper is included).

Local, three terminals

```
# 1. Presence service
cd presence-service && npm install && node src/index.js

# 2. Backend (MySQL). Defaults are baked in, so no env file is required.
cd backend && ./mvnw spring-boot:run

# 2b. Or with a throwaway in-memory database
cd backend && ./mvnw spring-boot:run -Dspring-boot.run.profiles=h2

# 3. Frontend
cd frontend && npm install && npm run dev
```

Then open `http://localhost:5173` and sign in as `faculty@smartattend.edu / Demo@123`.

If you prefer to supply credentials through `backend/.env`, **quote the values**. The JDBC URL contains `&`, which a shell interprets as a background operator, and an unquoted line produces `parse error near '&'` when sourced.

Docker

```
docker compose up --build
```

One caveat with real detection under Docker: a container has its own network namespace, so `arp -a` inside it shows the container's ARP table, not the classroom's. For genuine scanning, run the presence service on the host (or on the access point) and point the backend at it with `PRESENCE_WS_URL` and `PRESENCE_REST_URL`.

Useful URLs

URL	What
<code>http://localhost:5173</code>	Application
<code>http://localhost:8080/swagger-ui.html</code>	Interactive API docs
<code>http://localhost:8080/h2-console</code>	H2 console (h2 profile only)
<code>http://localhost:3001/presence/health</code>	Scanner status
<code>http://localhost:3001/presence/wireless</code>	Which AP the scanner detected, and how

16. Testing

`backend/src/test/java/.../PresenceConfidenceTest.java` — 11 JUnit 5 tests covering the scoring engine:

Test	Asserts
<code>deviceOnClassroomApWithStrongSignalIsHigh</code>	Correct AP plus strong signal reaches HIGH
<code>networkPresenceWithoutApProofCannotReachHigh</code>	The key invariant: no AP data can never be HIGH on a room-bound session
<code>deviceOnDifferentApIsRejectedAsLow</code>	Wrong AP drops below the refusal threshold
<code>weakSignalOnCorrectApIsPenalized</code>	Weak signal scores strictly lower than strong
<code>unboundSessionStillUsesNetworkOnlyScoring</code>	Legacy behaviour preserved when no room is bound
<code>unknownMacOutsideSubnetIsLow</code>	Unapproved and off-subnet is LOW
<code>ipChangeAndDuplicateMacPenalize</code>	Clone heuristics reduce the score
<code>bssidComparisonIgnoresCaseAndSeparators</code>	<code>a4-83-e7-...</code> equals <code>A4:83:E7:...</code>
<code>cidrMatchWorks</code>	Subnet arithmetic is correct
<code>manualMarkIsMedium</code>	Faculty override is 55 / MEDIUM
<code>levelBoundaries</code>	75 → HIGH, 50 → MEDIUM, 49 → LOW

```
cd backend && ./mvnw test
```

Testing the scoring engine is cheap precisely because it is a pure function — the reason it was written without Spring dependencies. The rest of the system is verified end to end by running the stack and driving the simulator.

17. The demo script

The most persuasive three minutes, in order:

1. Sign in as faculty (`faculty@smartattend.edu` / `Demo@123`).
2. **Sessions** → course name → select **CNT Lab 401** — **A4:83:E7:C0:FF:EE** → start. Point out the warning text shown when a room is *not* selected.
3. On the command center, press ► **Run demo classroom**.
4. Watch eight students appear as green blips, positioned by signal strength, each `HIGH (100)`.
5. Then show the red table: two students **seen on the network but not marked**, with the reason *"Associated with a different access point — not this classroom."*

Step 5 is the whole argument. Those two students are approved, on the campus subnet, and freshly seen in the ARP table — every signal a naive system checks. They are refused because they are on the neighbouring AP. Say out loud that a system without proximity proof would have marked them present.

Follow-ups worth having ready:

- **Admin** → **Rooms** shows the AP registry and the per-room dBm cutoff.
- `curl http://localhost:3001/presence/wireless` shows how the scanner learned the BSSID, including the `unavailable` fallback on macOS.
- The backend console prints the full reason list for every decision.

18. Design decisions and tradeoffs

MAC as identity, not IP. DHCP leases rotate; MAC is stable per NIC. IP is retained as corroborating evidence rather than identity.

Admin approval before detection. Any student could otherwise claim an arbitrary MAC, including a classmate's. Approval puts a human at the identity boundary. The one-approved -device rule stops device flooding.

Scoring as a pure function. No Spring, no database, no ambient clock — `now` is an input. This makes exhaustive unit testing trivial and keeps the security-critical logic readable in one screen.

Two scoring modes instead of one. Adding proximity as just another positive signal would have left the old maximum intact, and network-only evidence would still have reached HIGH. Rebalancing the weights when a room is bound is what makes AP association *necessary* rather than merely nice. Unbound sessions keep the old weights so nothing regresses.

Refusing rather than recording low confidence. Covered in section 7: a stored row acquires the appearance of fact regardless of its score.

Never fabricating an RSSI. When only the scanner's own signal is known, per-device RSSI stays `null` and the radar visibly treats it as unknown. A plausible-looking invented number would be worse than an honest gap.

Disconnects do not un-mark. Phones sleep their radios; attendance should not evaporate because a screen locked.

Most-recently-started active session wins. A pragmatic stand-in for real enrollment. Without a student-to-course mapping there is no principled way to choose among concurrent sessions, and the 10-minute cross-session guard limits the damage. A proper `enrollments` table is the correct fix and the most valuable remaining piece of data modelling.

Idempotent seeding. Checking existence before every insert means the seeder is safe to run against a populated database — important once real data exists.

19. Known limitations and threat model

An honest account of what this system cannot do. For a course report, this section is often worth more than another feature, because it demonstrates you understand the boundaries of your own design.

Detection limitations

1. **Passive ARP is incomplete.** The cache only lists hosts the scanner has recently exchanged frames with, so many classroom devices never appear. An active ARP sweep or ping sweep is needed for reliable coverage.
2. **ARP cache aging** makes "present now" imprecise, mitigated but not solved by the 30-second recency window.
3. **One broadcast domain per scanner.** Devices behind a router are invisible; a campus needs one scanner per VLAN.
4. **IPv4 only.** The parsers handle IPv4. Modern devices also use IPv6 with SLAAC and privacy extensions (RFC 4941) that rotate addresses, where NDP replaces ARP entirely. Using MAC as the identity key means this does not break the design, but the scanner sees less than it could.
5. **Per-client RSSI needs AP-mode access.** In observer mode you get room-level AP binding but no individual distances, so the radar falls back to its unknown ring.
6. **macOS cannot easily report BSSID.** `wdutil` redacts it without sudo and `airport` was removed in 14.4, so a Mac scanner typically reports `unavailable` and detection degrades to network-only scoring.

Security threats

See section 13 for the full list. The two most serious, ranked by how easily an undergraduate could exploit them:

- **MAC spoofing for proxy attendance.** Trivial with one command, and duplicate detection is currently inert.
- **Forged presence events.** The unauthenticated scanner-to-backend channel accepts attendance claims from anyone who can reach port 3002.

The real-world blocker: MAC randomisation

iOS 14+ and Android 10+ randomise their MAC address per SSID **by default**. A randomised address is stable for a given network, so registration still works in practice, but if a student toggles the setting or the OS rotates the address, the registered MAC stops matching and detection silently fails.

Three responses, in increasing rigour: instruct students to disable "Private Address" for the campus SSID; rely on the per-SSID stability of modern implementations and re-register when it changes; or abandon MAC as identity and move to **802.1X with WPA2-Enterprise**, where RADIUS Accounting-Start and Accounting-Stop packets provide cryptographically authenticated identity along with the access point and session times. The third is how a real campus would build this, and it is the most valuable direction for future work.

Correctness gaps

- No enrollment model, so session selection is heuristic.
- Sessions must be ended manually; nothing auto-closes them.
- Attendance is a single instant, not a duration, so leaving after five minutes is indistinguishable from staying the whole lecture.
- Attendance percentage divides by *all* sessions in the database rather than the sessions a student was enrolled in, which understates percentages in a multi-course dataset.
- `getSessionAttendance` uses `findAll()` and filters in memory, which is fine at demo scale and wrong at institutional scale.
- No NTP/clock-skew handling, although all timestamps come from one server so this only matters in a distributed deployment.

20. File-by-file map

presence-service/ — Node.js scanner

File	Contents
src/index.js	Scan loop, diffing, WebSocket server, REST API, simulator
src/scanner.js	arp -a execution and per-OS parsing; disconnect detection
src/wireless.js	BSSID/SSID/RSSI detection across macOS, Linux, Windows; iw station dump
src/config.js	Ports, scan interval, demo BSSIDs
Dockerfile	Container image

backend/ — Spring Boot

Path	Contents
SmartAttendanceApplication.java	Entry point
model/	User, Device, Classroom, Session, AttendanceRecord, Role
repository/	Five Spring Data interfaces
presence/	PresenceConfidence, PresenceSightingStore, MacVendor
service/	Auth, Device, Classroom, Session, Attendance, PresenceIngestion, PresenceGateway
controller/	Auth, Student, Faculty, Admin, Report
config/	SecurityConfig, WebSocketConfig, CorsConfig, DemoDataSeeder
security/	JwtTokenProvider, JwtAuthenticationFilter, CustomUserDetailsService
dto/	ApiResponse, AuthResponse, LoginRequest, RegisterRequest, DeviceRequest, SessionRequest, ClassroomRequest
resources/application*.properties	Default, MySQL and H2 configuration
src/test/	PresenceConfidenceTest

frontend/ — React

Path	Contents
src/App.jsx	Providers, routing, guards
src/api/api.js	Axios instance, interceptors, all endpoint wrappers
src/context/	AuthContext, ToastContext
src/pages/	Landing, Login, Register, Student x2, Faculty x2, Admin x3, Reports
src/components/	PresenceRadar, ConfidenceBadge, AttendanceHeatmap, PercentRing, CommandPalette, Navbar, ProtectedRoute
src/utils/attendanceMath.js	Pure helpers
src/index.css	All styling

Root

`docker-compose.yml` , `README.md` , `docs/api-contract.md` , this file.

21. Networking glossary

Term	Meaning in this project
ARP	Address Resolution Protocol (RFC 826). Maps an IPv4 address to a MAC address within one broadcast domain. The original detection mechanism.
ARP cache	The OS table of recent IP-to-MAC mappings, read with <code>arp -a</code> . Entries expire, which is why recency is scored.
MAC address	48-bit Layer 2 hardware address, written <code>AA:BB:CC:11:22:01</code> . The identity key for a student device.
OUI	Organizationally Unique Identifier — the first three octets of a MAC, assigned to a manufacturer. <code>MacVendor</code> uses it for vendor hints.
BSSID	MAC address of an access point radio. Because APs live in rooms, it functions as a room identifier.
SSID	The human-readable network name, e.g. <code>CAMPUS-WIFI</code> . Many APs share one SSID, which is why BSSID is the useful field.
RSSI	Received Signal Strength Indicator, in dBm and always negative. -45 is close, -85 is far. Drives radar distance and the room cutoff.
dBm	Decibels relative to one milliwatt. A logarithmic power scale.
Broadcast domain	The set of hosts reachable by a Layer 2 broadcast — the scanner's visibility limit.
CIDR	Notation like <code>192.168.0.0/16</code> describing a subnet by prefix length. Used for the campus-subnet check.
RFC 1918	The private address ranges <code>10/8</code> , <code>172.16/12</code> , <code>192.168/16</code> , the default campus subnets.
DHCP	Assigns IP addresses dynamically, which is why IP is unreliable as identity. Option 55 fingerprinting is a proposed anti-spoofing measure.
NDP	Neighbor Discovery Protocol, the IPv6 replacement for ARP.
SLAAC	IPv6 stateless address autoconfiguration; with privacy extensions (RFC 4941) addresses rotate.
MAC randomisation	Per-SSID randomised addresses on iOS 14+/Android 10+. The main real-world threat to this approach.
802.1X	Port-based network access control used by WPA2-Enterprise, giving authenticated identity via RADIUS. The rigorous alternative to MAC matching.
RADIUS	AAA protocol whose Accounting-Start/Stop records would provide authenticated attendance with AP and timing.
STOMP	Simple Text Oriented Messaging Protocol, run over WebSocket to give topic-based subscriptions to browsers.
SockJS	WebSocket emulation layer with fallbacks, used under STOMP.
JWT	JSON Web Token. Signed, stateless credential carrying the user's email as subject.
BCrypt	Adaptive salted password hashing function.